

Aproximación 3 - Práctica Final



Grado en Ingeniería Informática

Fecha de entrega: 18 / 03 / 2026

Nombre	NIA	Grupo	Equipo
Raoul Vlad Ivaszuk Ivaszuk	100508621	85	509
Jorge Lopez Alonso	100495876	85	509

Índice

1. Proceso de Adaptación	1
2. Batería de Pruebas	4

1. Proceso de Adaptación

Tras leer el ejercicio de Árboles de Sintaxis Abstracta hemos determinado que al código del ejercicio anterior teníamos que añadir las funciones que se describen en este. Para ello, lo primero que hemos hecho es añadir la siguiente estructura, que se encarga de almacenar la información de los nodos:

```
C/C++
// Estructura del nodo del Árbol de Sintaxis Abstracta (AST)
typedef struct ASTnode t_node;
struct ASTnode {
    char *op;
    int type; // 0: Hoja, 1: Unario, 2: Binario
    t_node *left;
    t_node *right;
};
```

También tuvimos que añadir una función que se encargue de crear estos nodos y otra de liberarlos/eliminarlos:

```
C/C++
// Función para crear un nuevo nodo
t_node * createASTNode (char *op, int type, t_node *left, t_node *right){
    t_node *node = (t_node*) malloc (sizeof (t_node));
    node->op = strdup (op);
    node->type = type;
    node->left = left;
    node->right = right;
    return node;
}
// Función para liberar la memoria del AST
void freeAST (t_node *node) {
    if (node != NULL) {
        freeAST (node->left);
        freeAST (node->right);
        free (node->op);
        free (node);
    }
}
```

Y por último para que se imprima por pantalla la solución hemos adoptado la función del enunciado que se encarga de esto mismo:

```
C/C++
// Función para imprimir el AST en notación prefija
```

```
void printAST2Prefix (t_node *node) {
    if (node == NULL) return;
    if (node->type == 0) {
        // Nodo Hoja
        printf("%s", node->op);
    } else if (node->type == 1) {
        // Nodo Unario
        printf("(%s ", node->op);
        printAST2Prefix(node->left);
        printf(")");
    } else {
        // Nodo Binario
        printf("(%s ", node->op);
        printAST2Prefix(node->left);
        printf(" ");
        printAST2Prefix(node->right);
        printf(")");
    }
}
```

Además a la estructura de la pila de bison hemos añadido el atributo `node`.

Por último, para tratar con las acciones semánticas de nuestra gramática hacemos llamadas a la función primitiva de creación de nodos para ir creando el árbol poco a poco. Estos son los cambios que ejecutado respecto al punto de partida:

- **Propagación de Nodos en lugar de Cadenas:** En todas las reglas de transición directa (como `sentencia: expresion, expresion: termino`, o el paso de expresiones entre paréntesis), hemos sustituido la asignación del atributo `$$cadena = $1.cadena` por la propagación de punteros a los subárboles mediante el nuevo atributo `$$node = $1.node`.
- **Creación de Nodos Hoja (Operandos):** Hemos modificado las reglas terminales correspondientes a `NUMERO` y `VARIABLE`. Ahora, al detectar uno de estos tokens, instanciamos un nodo hoja (tipo 0) mediante la función `createASTNode`, pasando como valor el literal numérico o el identificador de la variable (previamente convertidos a cadena) y dejando sus hijos izquierdo y derecho a `NULL`.
- **Operaciones Aritméticas (Nodos Binarios y Unarios):** Para las operaciones como la suma, resta, multiplicación y división, hemos eliminado la reserva de memoria temporal y el uso de `sprintf`. En su lugar, cada operación genera un nodo binario (tipo 2) donde la raíz contiene el operador aritmético y los atributos `$1.node` y `$3.node` se enlazan como subárboles izquierdo y derecho respectivamente. Para el caso del menos unario, generamos un nodo de tipo 1 con un único hijo izquierdo.

- **Traducción de Asignaciones y Encadenamiento:** En la regla de asignación, hemos traducido el símbolo `=` por la función `setq` de *Lisp*. Para lograrlo, creamos un nodo hoja para la variable destino y un nodo binario "`setq`" que enlaza dicha variable con la expresión evaluada. Adicionalmente, para cumplir con las especificaciones avanzadas del enunciado, hemos introducido una nueva producción recursiva (*VARIABLE '=' asignacion*). Dado que el operador `=` tiene asociatividad por la derecha (*%right*), esta nueva regla nos permite soportar asignaciones encadenadas del tipo `a=b=c=2+3`, anidando correctamente los nodos `setq` de derecha a izquierda en el AST de forma natural.
- **Sentencia de Impresión:** La regla de impresión (*@ expresion*) se ha actualizado para generar un nodo unario (tipo 1) etiquetado con la función "`print`", cuyo único hijo es el subárbol de la expresión que se desea imprimir.
- **El Axioma: Evaluación y Gestión de Memoria:** Finalmente, hemos intervenido en la cima de la gramática. Cuando el axioma reconoce una sentencia completa terminada en un salto de línea (`\n`), tomamos el nodo raíz resultante (`$2.node`) y ejecutamos la función `printAST2Prefix` para volcar la traducción en notación prefija de *Lisp* a la salida estándar. Inmediatamente después, para asegurar una gestión eficiente y evitar fugas de memoria a medida que el intérprete procesa múltiples líneas, hemos incorporado una llamada a `freeAST($2.node)`, destruyendo el árbol generado tras su utilización.

2. Batería de Pruebas

Para comprobar el funcionamiento correcto de nuestro fichero, Hemos optado por llevar a cabo las siguientes pruebas:

Pruebas.txt

```
None
1+2*3
-5+2
a=1
b=2
(a+b)*3
x=(2+3)*(4-1)
@ (x+10)/2
```

Para luego usar clisp, previamente guardamos la salida en el fichero pruebas.1, Obteniendo estos resultados correctamente:

```
None
(+ 1 (* 2 3))
(+ (- 5) 2)
(setq a 1)
(setq b 2)
(* (+ a b) 3)
(setq x (* (+ 2 3) (- 4 1)))
(print (/ (+ x 10) 2))
```

Por último aplicando clisp de pruebas.1, el resultado que se obtiene por pantalla es **25/2**, resultado de $(/ (+ x 10) 2)$.